

МИНОБРНАУКИ РОССИИ

---

Федеральное государственное бюджетное  
образовательное учреждение высшего образования  
«Омский государственный технический университет»

# РАЗРАБОТКА АРИФМЕТИЧЕСКИХ УСТРОЙСТВ

*Учебное текстовое электронное издание  
локального распространения*

Омск  
Издательство ОмГТУ  
2017

Составитель *И. В. Червенчук*, канд. техн. наук, доцент

**Разработка арифметических устройств** : метод. указания / Минобр-науки России, ОмГТУ ; [сост. И. В. Червенчук]. – Омск : Изд-во ОмГТУ, 2017.

Изложены основы двоичной компьютерной арифметики, приводятся примеры алгоритмов выполнения арифметических операций в ЭВМ для двоичных чисел в формате с плавающей запятой. Рассмотрены методы реализации основных арифметических операций посредством цифровых устройств. Описаны алгоритмические модели выполнения арифметических операций и изложены методы их проектирования.

Методические указания предназначены для студентов, обучающихся по направлению «Информатика и вычислительная техника».

*Рекомендовано редакционно-издательским советом  
Омского государственного технического университета*

© ОмГТУ, 2017

1 электронный оптический диск

Оригинал-макет издания выполнен в Microsoft Office Word 2007/2010 с использованием возможностей Adobe Acrobat Reader.

**Минимальные системные требования:**

- процессор Intel Pentium 1,3 ГГц и выше;
- оперативная память 256 Мб и более;
- свободное место на жестком диске 260 Мб и более;
- операционная система Microsoft Windows XP/Vista/7/10;
- разрешение экрана 1024×768 и выше;
- акустическая система не требуется;
- дополнительные программные средства Adobe Acrobat Reader 5.0 и выше.

Редактор *Т. А. Москвитина*  
Компьютерная верстка *Л. Ю. Бутаковой*

Сводный темплан 2017 г.  
Подписано к использованию 20.06.17.  
Объем 0,4 Мб.

---

Издательство ОмГТУ.  
644050, г. Омск, пр. Мира, 11; т. 23-02-12  
Эл. почта: [info@omgtu.ru](mailto:info@omgtu.ru)

## ПРЕДИСЛОВИЕ

В учебных планах подготовки бакалавров по направлению «Информатика и вычислительная техника» дисциплина «Арифметические и логические основы вычислительной техники» входит в состав фундаментального цикла дисциплин, формирующих у студентов представление о базовых понятиях информатики и вычислительной техники как о предмете их дальнейшей профессиональной деятельности. Изучение указанной дисциплины и овладение навыками разработки и моделирования алгоритмов выполнения арифметических и логических операций, синтеза логических схем, разработки устройств на структурном уровне создает теоретическую базу для изложения таких дисциплин, как «Прикладная теория цифровых автоматов», «Схемотехнические основы вычислительных систем», «Микропроцессорные системы» и др.

Краткий теоретический материал, изложенный в методических указаниях, поможет в выполнении расчетно-графической работы по дисциплине «Арифметические и логические основы вычислительной техники», а также курсового проекта (на первом этапе) по дисциплине «Прикладная теория цифровых автоматов».

Итогом расчетно-графической работы по дисциплине «Арифметические и логические основы вычислительной техники» является разработка устройства, выполняющего заданную арифметическую операцию, предполагающей разработку структуры устройства на базе типовых цифровых узлов и разработку управляющего алгоритма данного устройства.

Также приводятся примеры разработки подобных арифметических устройств.

# 1. ОСНОВЫ ДВОИЧНОЙ КОМПЬЮТЕРНОЙ АРИФМЕТИКИ

## 1.1. ПОЗИЦИОННЫЕ СИСТЕМЫ СЧИСЛЕНИЯ И ВЫПОЛНЕНИЕ АРИФМЕТИЧЕСКИХ ОПЕРАЦИЙ

В современных вычислительных системах для представления обрабатываемой числовой информации используются, как правило, позиционные системы счисления.

Системой счисления называется совокупность принципов и правил, позволяющих представить число.

Позиционной системой счисления называют такую систему, в которой значение (вес) каждой цифры, образующей число, зависит от ее позиции в числе. То есть одна и та же цифра в зависимости от ее местоположения в числе умножается на различный весовой коэффициент для определения ее значения.

Основанием системы счисления называется количество различных цифр, применяемых в соответствующей позиционной системе счисления, необходимых для изображения любого числа в этой системе.

Так, например, основанием двоичной позиционной системы является число 2, восьмеричной – 8, десятичной – 10, шестнадцатеричной – 16 и т. д.

Пусть  $p$  ( $p = 2, 3, \dots$ ) является основанием  $p$ -ичной позиционной системы счисления. В такой системе используется  $p$  различных цифр, представленное множеством  $\{0, 1, 2, \dots, p-1\}$ . Любые числа записываются в виде последовательности цифр, в которых целая часть отделена от дробной части запятой (точкой) и каждая последующая (справа налево) цифра имеет весовой коэффициент в  $p$  раз больше, чем предыдущая.

Если буквами  $\alpha_n, \alpha_{n-1}, \dots, \alpha_1, \alpha_0, \alpha_{-1}, \alpha_{-2}, \dots, \alpha_{-m}$  обозначить цифры  $p$ -ичной позиционной системы счисления, то последовательность цифр обозначает число  $\alpha_n \alpha_{n-1} \dots \alpha_1 \alpha_0 \alpha_{-1} \alpha_{-2} \dots \alpha_{-m}$ .

$$A_{(p)} = \alpha_n p^n + \alpha_{n-1} p^{n-1} + \dots + \alpha_1 p^1 + \alpha_0 p^0 + \alpha_{-1} p^{-1} + \alpha_{-2} p^{-2} + \dots + \alpha_{-m} p^{-m}.$$

Преимуществом позиционных систем счисления является приспособленность их к выполнению арифметических операций, арифметические операции можно выполнять последовательно, начиная с младших или старших цифр числа, а не над всем числом в целом неразрывно.

В двоичной системе счисления для записи произвольных чисел используются цифры из множества  $\{0,1\}$ . Основание этой системы счисления число два ( $p = 2$ ). (Вопросы перевода из двоичной системы в десятичную здесь рассматриваться не будут, подробнее см. учебное пособие [3].)

Например, десятичное число 173,75 в двоичной системе счисления будет выглядеть так:

$$10101101,11 = 1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2}.$$

Легко проверить теперь правильность двоичной записи данного числа, переписав правую часть последнего равенства в десятичной системе счисления:

$$1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = \\ = 128 + 0 + 32 + 0 + 8 + 4 + 0 + 1 + 0,5 + 0,25 = 173,75.$$

Таким образом, весовые коэффициенты цифр двоичного числа являются числами, кратными целой степени (положительной или отрицательной) цифры 2. При этом для целой части двоичного числа, начиная от младшего разряда к старшему, они равны 1, 2, 4, 8, 16, 32, 64, 128 и т. д., а для дробной части двоичного числа, начиная от старшего разряда к младшему, весовые коэффициенты цифр имеют значения  $1/2$ ,  $1/4$ ,  $1/8$ ,  $1/16$ ,  $1/32$ ,  $1/64$ ,  $1/128$  и т. д.

Таблицы сложения и умножения двоичных цифр, лежащие в основе двоичной арифметики, содержат всего по четыре строки, что существенно облегчает техническую реализацию машинных операций.

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

Двоичная  
таблица  
сложения

$$0 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$

$$1 \cdot 0 = 0$$

$$1 \cdot 1 = 1$$

Двоичная  
таблица  
умножения

На базе приведенных таблиц арифметических операций выполняются арифметические операции сложения, умножения, деления.

### Пример 1.

#### Сложение

$$\begin{array}{r} 11001011,101 \\ + 1010101,111 \\ \hline 100100001,100 \end{array}$$

#### Умножение

$$\begin{array}{r} 10110101,011 \\ \times 10,101 \\ \hline 10110101,011 \\ + 10110101,011 \\ \hline 10110101,011 \\ \hline 111011100,000111 \end{array}$$

#### Деление

$$\begin{array}{r|l} 111011100000111 & 10110101011 \\ - 10110101011 & 10101 \\ \hline 0011100010101 \\ - 10110101011 & \\ \hline 0010110101011 \\ - 10110101011 & \\ \hline 0 \end{array}$$

Реализация указанных операций в цифровых вычислительных устройствах имеет свои особенности, которые будут рассмотрены в дальнейшем.

## 1.2. ПРЕДСТАВЛЕНИЕ ЧИСЕЛ С ПЛАВАЮЩЕЙ ЗАПЯТОЙ (ТОЧКОЙ)

Формат чисел с плавающей запятой позволяет представить вещественные числа в большом диапазоне значений и применяется для них наиболее часто. Любое число в позиционной системе счисления можно представить в следующем виде:

$$N = \pm m \cdot 10^{\pm p},$$

где  $m$  – мантисса числа  $N$ , причем  $0,5 \leq |m| < 1,0$  (или в двоичной системе  $0,1 \leq |m| < 1,0$ ;  $p$  – нуль или произвольное целое число, называемое порядком числа  $N$ ;  $10 (2_{10})$  – основание системы счисления.

Например, при двоичной записи мантиссы  $m$  и порядка  $p$  имеем:

$$N_1 = + 0,100101 = + 0,100101 \cdot 10^0; (m_1 = + 0,100101; p_1 = 0);$$

$$N_2 = - 1001,01 = - 0,100101 \cdot 10^{+100}; (m_2 = - 0,100101; p_2 = +100 = 4);$$

$$N_3 = + 0,000101 = + 0,101000 \cdot 10^{-11}; (m_3 = + 0,101000; p_3 = -11 = -3).$$

Из приведенных примеров легко видеть, что мантисса  $m$  представляет собой цифровую часть числа, выраженную правильной дробью, а порядок  $p$

указывает на место положения запятой в числе  $N_i$  и является целым числом. То есть порядок  $p$  указывает номер разряда правильной дроби  $m$ , после которого при  $p > 0$  или перед которым при  $p < 0$  необходимо поставить запятую, начиная счет с первого (старшего) разряда дроби.

Подобный вид записи числа называется нормализованным. Будем считать, что в качестве входных операндов разрабатываемых устройств будут подаваться числа в нормализованной форме, при этом и на само устройство накладывается требование выдавать данные в нормализованном виде.

### 1.3. ПРЕДСТАВЛЕНИЕ ОТРИЦАТЕЛЬНЫХ ЧИСЕЛ

Для представления отрицательных чисел используется прямой, обратный и дополнительный коды.

В прямом коде первый разряд кодирует знак числа: 0 – число положительное, 1 – число отрицательное, остальные разряды числа остаются без изменения. Обратный код получается путем инверсии всех разрядов числа, таким образом, старший разряд будет содержать 1. Дополнительный код дополняет исходное число до переполнения разрядной сетки таким образом, что при сложении такого числа с исходным получим число 0,0000...0 и перенос из старшего разряда. Получить дополнительный код можно путем инверсии всех разрядов исходного числа с прибавлением единицы к младшему разряду, или инвертируя все разряды исходного числа, начиная со старших до последней единицы, последнюю единицу оставляем. Если применить процедуру вычисления дополнительного кода дважды, вернемся к исходному числу. В следующем примере приводится прямой, обратный и дополнительный коды числа.

#### Пример 2.

$$\begin{array}{rcl}
 X = -0,10110100 & & 0,10110100 \\
 [X]_{\text{пр}} = -1,10110100 & & + \quad \underline{1,01001100} \\
 [X]_{\text{обр}} = -1,01001011 & & 0,00000000 \\
 [X]_{\text{доп}} = -1,01001100 & & 
 \end{array}$$

Для положительного числа прямой, обратный и дополнительный коды совпадают.

Для непосредственной реализации операций наиболее удобен дополнительный код, при выполнении операций в котором не требуется каких-

либо дополнительных действий, поэтому в большинстве систем используется именно дополнительный код для представления отрицательных чисел. При этом для реализации вычитания второй операнд переводится в дополнительный код, то есть:

$$A - B = A + [B]_{\text{доп.}}$$

При этом не важно, какой знак имеет операнд В, если он положительный, то будет представлен отрицательным числом в дополнительном коде, если он отрицательный, то станет положительным.

### Пример 3.

$$\begin{array}{lll} A = 0,10101011 & B = 0,11001101 & A = 0,10101011 \\ -A = 1,01010101 & -B = 1,00110011 & \begin{array}{r} + \\ -B = 1,00110011 \\ \hline A - B = 1,11011110 \end{array} \end{array}$$

В примере 3 ответ получился отрицательный.

При выполнении операций в прямом коде сначала нужно проанализировать знаки операндов, выбрать необходимые операции, занулить знаки операндов и при необходимости принудительно присвоить знак результату. Например, при сложении чисел в прямом коде  $A$  и  $B$  ( $A > 0$ ,  $B < 0$ ) нужно будет сделать следующее: занулить знак  $B$ , перевести в дополнительный код, сложить с  $A$ , если ответ получится отрицательный, перевести его обратно в прямой код и присвоить знаковому разряду 1.

При выполнении операций в обратном коде необходимо использовать правило: при появлении переноса из старшего разряда добавляется 1 в младший разряд.

### Пример 4.

$$\begin{array}{lll} A = 0,10101011 & B = 0,11001101 & -A = 1,01010100 \\ -A = 1,01010100 & -B = 1,00110010 & \begin{array}{r} + \\ B = 0,11001101 \\ \hline 1\_0,00100001 \\ \hline \phantom{1\_}1 \\ B - A = 0,00100010 \end{array} \end{array}$$

## 1.4. ПЕРЕПОЛНЕНИЯ ПРИ ВЫПОЛНЕНИИ ОПЕРАЦИЙ

При выполнении некоторых операций, например сложении операндов одного знака, могут возникать переполнения, которые необходимо отслеживать. Признаком переполнения при сложении двух положительных чи-

сел является появление 1 в знаковом разряде суммы, при сложении двух отрицательных чисел – появление 0 в знаковом разряде суммы. При сложении двух чисел разных знаков переполнения невозможны.

Чтобы легче отследить переполнения, используются так называемые модифицированные коды, использующие два разряда для кодирования знака. Положительное число кодируется как 00,1XX...X, отрицательное – как 11,0...., для нормализованного числа знаковый разряд должен отличаться от старшего разряда.

#### Пример 5.

$$\begin{array}{rcl}
 + A = 00,10101011 & & + -A = 11,01010101 \\
 + B = 00,11001101 & & + -B = 11,00110011 \\
 \hline
 A + B = 01,01111000 & & -A - B = 10,10001000
 \end{array}$$

При сложении  $A + B$  в примере 5 получаем положительное число с переполнением (01, – признак положительного переполнения), при сложении  $(-A) + (-B)$  получаем отрицательное число с переполнением (10, – признак отрицательного переполнения). И в том, и в другом случае при использования формата с плавающей запятой нужно нормализовать результат путем правого сдвига с сохранением знака (в сторону младших разрядов) и увеличения на единицу порядка результата.

Если в результате сложения получаем число с комбинацией знаковых разрядов 00, – число положительное без переполнения, 11, – число отрицательное без переполнения,

Модифицированные коды более удобны при выполнении операций, особенно при отслеживании возникающих переполнений, но требуют дополнительных аппаратных затрат.

## 2. ОСНОВЫ РАЗРАБОТКИ АРИФМЕТИЧЕСКИХ УСТРОЙСТВ

### 2.1. БАЗОВЫЕ ЦИФРОВЫЕ УЗЛЫ

Арифметические операции в вычислительных системах реализуются на арифметико-логических устройствах, которые строятся на основе типовых цифровых узлов: регистров, счетчиков сумматоров. Рассмотрим данные типовые цифровые узлы.

Элементной базой для построения цифровых узлов являются логические элементы и триггеры.

Триггер – бистабильная ячейка памяти, которая может находиться в одном из двух устойчивых состояний и способна хранить один бит информации.

Существует достаточно разветвленная классификация триггеров: по уравнению функционирования, по способу управления, по элементной базе и т. д., но в данном случае будем считать, что триггер сохраняет один бит информации, поданный на его вход под воздействием управляющего сигнала загрузки триггера, и загруженный в триггер бит информации всегда доступен для прочтения.

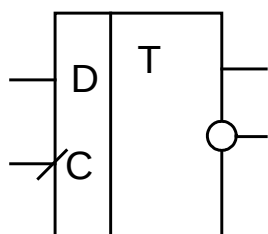


Рис. 1. Триггер

На рис. 1 изображен D-триггер с информационным входом D и управляющим C.

Цепочка триггеров образует регистр. Регистр – устройство, реализующее функции сверхоперативной памяти. Обычно в регистрах хранятся оперативные данные, которые используются непосредственно при выполнении

операции. Кроме того, на регистрах выполняются операции сдвига и преобразование данных из последовательного в параллельный код и наоборот. При выполнении арифметических операций в регистрах хранятся операнды и промежуточные данные, также на них реализуются сдвиги: сдвиг в сторону младших разрядов соответствует делению числа на два, сдвиг в сторону старших разрядов – умножению на два.

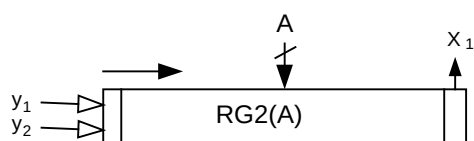


Рис. 2. Регистр

На рис. 2 приводится условное изображение регистра с именем RG2, в который загружается операнд A и который может выполнить сдвиг в сторону младших разрядов. Стрелка, идущая от операнда A к регистру,

перечеркнута, это означает, что шина передачи данных от А к регистру является могоразрядной. На регистр подаются два управляющих сигнала (например,  $y_1$  – загрузка,  $y_2$  – сдвиг вправо). С младшего разряда регистра снимается логическое условие  $x_1$  ( $x_1 = 0$ , если младший разряд регистра равен 0, или  $x_1 = 1$ , если младший разряд регистра равен 1), которое может анализироваться алгоритмом обработки.

Сумматор – устройство, реализующее операцию сложения. При реализации арифметических операций удобнее использовать так называемые накапливающие сумматоры. При выполнении операции сложения на таком сумматоре содержимое сумматора складывается с данными, поступающими на его вход. Кроме операции сложения сумматор должен выполнять команду сброса, обнуляющую сумматор. Накапливающий сумматор содержит в своем составе регистр, соответственно может выполнять все операции, присущие регистрам, в частности сдвиги.

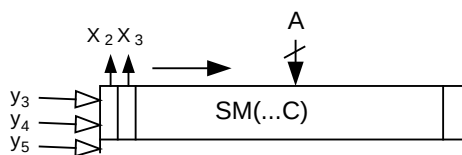


Рис. 3. Сумматор

На рис. 3 приводится условное изображение сумматора с именем SM, на шину которого подается операнд А, в котором по окончании выполнения операции будет находиться результат выполнения операции – С и который соответственно может выполнить операцию  $SM = SM + A$ . При этом сумматор также может выполнить сдвиг вправо. На сумматор подаются управляющие сигналы (например,  $y_3$  – сброс, то есть  $SM = 0$ ,  $y_4$  – суммирование,  $y_5$  – сдвиг вправо). Со старших разрядов сумматора снимаются логические условия  $x_2, x_3$ , позволяющие анализировать знак числа на сумматоре и его старший цифровой разряд, например, для проверки на предмет денормализации результата.

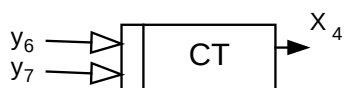


Рис. 4. Счетчик

Счетчик – устройство, осуществляющее подсчет импульсов, поступающих на его вход. В приведенном случае (рис. 4) на счетчик подается сигнал сброса  $y_6$  ( $СТ = 0$ ) и сигнал, увеличивающий его содержимое на единицу,  $y_7$  (инкремент). Кроме перечисленных сигналов на счетчик может подаваться сигнал загрузки (содержимое счетчика загружается подаваемыми на него внешними данными) и сигнал декремента, уменьшающего содержимое счетчика на единицу. Часто бывает удобнее загрузить начальное значение в счетчик, а потом подавать сигнал декремента, ожидая обнуление счетчика (которое сопровождается выдачей соответству-

ющего выходного сигнала). При разработке арифметических устройств счетчики используются для подсчета необходимого числа циклов.

На базе рассмотренных типовых узлов: регистров, сумматоров, счетчиков, в некоторых случаях с привлечением отдельных триггеров и логических элементов реализуется построение операционной части арифметико-логических устройств, реализующих арифметические операции.

## 2.2. ВЫПОЛНЕНИЕ ДЛИННЫХ ОПЕРАЦИЙ

Длинными операциями в машинной арифметике считаются операции умножения, деления, извлечения квадратного корня. Они называются длинными, поскольку для их реализации требуется выполнение ряда элементарных операций сдвига, суммирования, установки разрядов.

### Реализация операции умножения

Рассмотрим в качестве примера реализации операции умножения реализацию умножения двоичных чисел в формате с плавающей запятой в прямом коде с использованием алгоритма «младшими разрядами вперед» (он же алгоритм с подвижным сумматором). На рис. 5 приводится упрощенная структура устройства, реализующего выполнение данной операции. Устройство обработки мантисс содержит два регистра для хранения операндов и накапливающий сумматор с дополнительным разрядом для округления. Дополнительно имеются триггер для хранения знака результата и счетчик для подсчета необходимого числа циклов, определяющего количеством разрядов числа без учета знакового. Управляющие сигналы и часть устройства, выполняющего работу с порядками, не показаны.

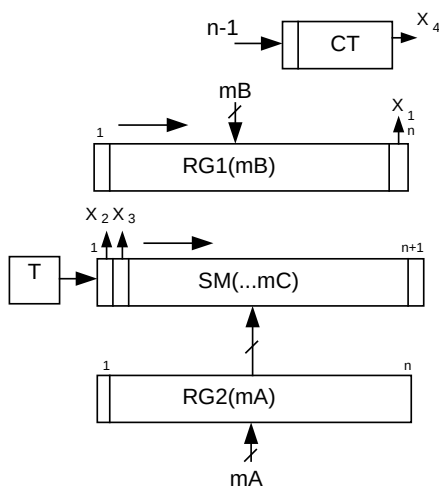


Рис. 5. Устройство умножения

Предполагается, что данные на вход устройства подаются в нормализованном виде. При выполнении операции  $C = A \times B$  мантисса  $A$  загружается в регистр RG2, мантисса  $B$  загружается в регистр RG1, сумматор SM обнуляется, в счетчик загружается число разрядов числа без учета знакового –  $n-1$ . При выполнении операции умножения порядки операндов должны складываться:  $pC = pA + pB$ , выполнение этой операции предполагается на некотором дополнительном сумматоре порядков (на рис. 5 не показан). Поскольку предполагается, что данные хранятся в прямом коде, то знаки операндов следует обработать отдельно и сохранить в триггере, то есть выполняется операция определения знака результата и сохранения его в триггере как

$$T = Z_n(A) \oplus Z_n(B),$$

где  $\oplus$  – логическая операция «сумма по модулю два»;

$Z_n(A)$  – знак  $A$  (первый разряд мантиссы  $mA$ );

$Z_n(B)$  – знак  $B$  (первый разряд мантиссы  $mB$ ).

После чего знаки операндов необходимо принудительно обнулить.

Собственно умножение реализуется следующим образом:

1. Анализируется младший разряд регистра RG1 (логическое условие  $x_1$ ), и если разряд равен 1, выполняется операция на сумматоре  $SM = SM + RG2$ , для обеспечения данной операции в структуре предусмотрена многоразрядная связь от RG2 к сумматору.

2. Выполняются сдвиги сумматора и RG1 в сторону младших разрядов:  $SM \rightarrow$ ,  $RG1 \rightarrow$ .

3. Содержимое счетчика уменьшается на единицу  $CT--$ , если  $CT \neq 0$ , продолжить с 1.

4. Выполняется округление  $SM = SM + 2^{-(n+1)}$ .

Округление, реализуемое путем прибавления единицы к младшему дополнительному разряду сумматора, желательно выполнять для того, чтобы избежать накопления систематической ошибки отрицательного знака.

По окончании указанных действий на сумматоре должен находиться результат. Перед выдачей результата необходимо выполнить проверку на нормализацию результата (посмотреть второй разряд сумматора – логическое условие  $x_3$ , он должен быть равен 1). Поскольку изначально операнды находятся в нормализованном виде, денормализация может быть не более чем на один знак. Если  $x_3 = 0$ , то необходимо перед выдачей выполнить нормализа-

цию, то есть сдвинуть сумматор в сторону старших разрядов, что эквивалентно умножению на 2, и порядок результата  $pC$  уменьшить на единицу.

Далее необходимо загрузить из триггера знак результата в первый разряд сумматора  $SM[1] = T$ , после чего в сумматоре  $SM$  находится мантисса результата операции умножения, предполагается, что порядок находится в специальном регистре или сумматоре  $pC$ .

Приведем пример выполнения данного алгоритма (пример 6).

В первом такте каждого цикла производится (или нет) операция сложения, а во втором такте, в любом случае, выполняется сдвиг в сторону младших разрядов.

### Пример 6.

$$mA = 0,10101101 \quad pA = 00001$$

$$mB = 0,11001001 \quad pB = 00011$$

$$T = 0 \oplus 0 = 0$$

$$SM = 0,00000000|0$$

$$RG2 = 0,10101101|0$$

$$1) \quad SM = 0,10101101|0 \quad RG1 = 0,11001001$$

$$SM \rightarrow = 0,01010110|1$$

$$2) \quad SM \rightarrow = 0,00101011|0 \quad RG1 \rightarrow = 0,01100100$$

$$3) \quad SM \rightarrow = 0,00010101|1 \quad RG1 \rightarrow = 0,00110010$$

$$4) \quad RG2 = 0,10101101|0 \quad RG1 = 0,00011001$$

$$SM = 0,11000010|1$$

$$SM \rightarrow = 0,01100001|0$$

$$5) \quad SM \rightarrow = 0,00110000|1 \quad RG1 = 0,00001100$$

$$6) \quad SM \rightarrow = 0,00011000|0 \quad RG1 = 0,00000110$$

$$7) \quad RG2 = 0,10101101|0 \quad RG1 = 0,00000011$$

$$SM = 0,11000101|0$$

$$SM \rightarrow = 0,01100010|1$$

$$8) \quad RG2 = 0,10101101|0 \quad RG1 = 0,00000001$$

$$SM = 1,00001111|1$$

$$SM \rightarrow = 0,10000111|1$$

$$\underline{\hspace{1.5cm}} \quad 1(\text{округление})$$

$$SM = 0,10001000|0$$

$$pA = 00001$$

$$pB = 00011$$

$$pC = 00100$$

$$\text{Ответ: } mC = 0,10001000 \quad pC = 00100.$$

## Реализация операции деления

Рассмотрим в качестве примера реализации операции деления реализацию деления двоичных чисел в формате с плавающей запятой в прямом коде с использованием алгоритма без восстановления остатка с неподвижным сумматором. На рис. 6 приводится упрощенная структура устройства, реализующего выполнение данной операции. Устройство обработки мантисс также содержит два регистра для хранения операндов и накапливающий сумматор, дополнительно имеются триггер для хранения знака результата и счетчик для подсчета необходимого числа циклов, определяющегося количеством разрядов числа без учета знакового. Управляющие сигналы и часть устройства, выполняющего работу с порядками, не показаны.

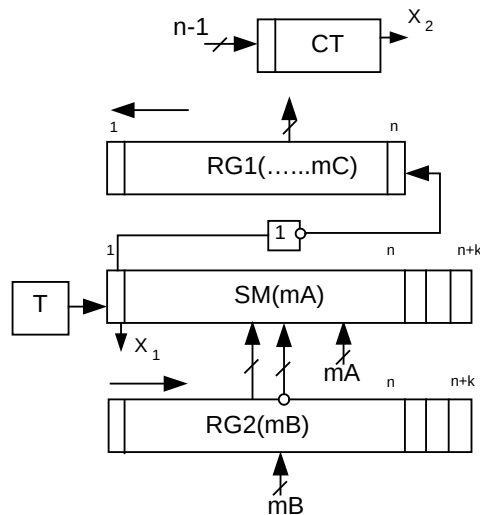


Рис. 6. Устройство деления

Предполагается, что данные на вход устройства подаются в нормализованном виде. При выполнении операции  $C = A/B$  мантисса  $A$  загружается в сумматор  $SM$ , мантисса  $B$  загружается в регистр  $RG2$ , регистр формирования результата  $RG1$  обнуляется, в счетчик загружается число разрядов числа без учета знакового –  $n-1$ . К сумматору от  $RG2$  подаются две многоразрядные связи, обеспечивающие выполнение двух операций на сумматоре:  $SM = SM + RG2$  и  $SM = SM - RG2$ . Знак сумматора инвертируется и задвигается в регистр результата  $RG1$ .

При выполнении операции деления порядки операндов должны вычитаться:  $pC = pA - pB$ , выполнение этой операции предполагается на некотором дополнительном сумматоре порядков (на рис. 6 не показан). Поскольку предполагается, что данные хранятся в прямом коде, то знаки операндов следует обработать отдельно и сохранить в триггере, то есть выполняется

операция определения знака результата и сохранения его в триггере, как при умножении:

$$T = Z_n(A) \oplus Z_n(B),$$

где  $\oplus$  – логическая операция «сумма по модулю два»;

$Z_n(A)$  – знак A (первый разряд мантиссы  $m_A$ );

$Z_n(B)$  – знак B (первый разряд мантиссы  $m_B$ ).

После чего знаки операндов необходимо принудительно обнулить.

При выполнении алгоритма данные будут сдвигаться в сторону младших разрядов, поэтому в сумматоре и связанном с ним регистре (RG2) необходимо предусмотреть дополнительные разряды. Общее число разрядов соответствует  $n + k$ , где число дополнительных разрядов  $k$  исходя из обеспечения приемлемой погрешности ( $\leq 2^{-n}$ ) выбирается в соответствии с выражением  $k \geq \log_2(n-k)$ , например, для  $n = 8$   $k = 3$ .

Исходя из необходимости получения нормализованного результата, перед выполнением непосредственно алгоритма деления необходимо сделать проверку на  $m_A < m_B$ . Данная проверка осуществляется с модулями операндов, при использовании прямых кодов после обнуления результатов на регистре и сумматоре как раз имеем соответственно модули операндов. Сравнение может быть выполнено с помощью специальной логической схемы сравнения либо алгоритмически путем вычитания. В последнем случае из сумматора SM вычитается содержимое регистра RG2, и если результат вычитания получается отрицательный (знаковый разряд сумматора – логическое условие  $x_1$  равен 1), то  $m_A < m_B$ , если положительный ( $x_1$  равен 0), то  $m_A < m_B$  не выполняется и необходимо будет выполнить денормализацию операнда A, находящегося на сумматоре. После выполнения вычитания  $m_A$  на сумматоре разрушается, и это можно компенсировать восстановлением сумматора, прибавив к нему RG2, или при использовании алгоритма без восстановления остатка учесть это и первый цикл начать со сложения на сумматоре. Денормализация заключается в сдвиге сумматора в сторону младших разрядов и увеличения порядка на единицу.

Собственно деление реализуется следующим образом:

1. Выполняется сдвиг RG2 в сторону младших разрядов  $RG2 \rightarrow$ .
2. Если сумматор SM положительный ( $SM > 0$ ), выполняется операция на сумматоре  $SM = SM - RG2$ , если сумматор отрицательный ( $SM < 0$ ), то  $SM = SM + RG2$ .

3. Инверсия знакового разряда сумматора задвигается в регистр формирования результата при выполнении левого сдвига RG1 (RG1←-).

4. Содержимое счетчика уменьшается на единицу CT--, если CT≠0, продолжить с 1.

По окончании указанных действий на RG1 должен находиться результат. Если производилась предварительная проверка и приведение к  $mA < mB$ , результат получается нормализованным.

Далее необходимо загрузить из триггера знак результата в первый разряд  $RG1[1] = T$ , после чего в RG1 находится мантисса результата операции умножения, предполагается, что порядок находится в специальном регистре или сумматоре pC.

Приведем пример (пример 7) выполнения данного алгоритма.

В первом такте каждого цикла производится сдвиг RG2 в сторону младших разрядов, во втором такте выполняется операция на сумматоре, в третьем такте – загрузка очередного разряда результата в RG1.

#### Пример 7.

$$\begin{aligned} mA &= 0,10110101 & pA &= 00101 \quad (5) \\ mB &= 0,11001001 & pB &= 10001 \quad (-1) \end{aligned}$$

$$mA < mB \quad T = 0 \oplus 0 = 0$$

|    |        |                       |        |            |
|----|--------|-----------------------|--------|------------|
|    | SM =   | 0,10110101 000        | RG1 =  | 0,00000000 |
|    | RG2 =  | 0,11001001 000        |        |            |
| 1) | RG2→=  | 0,01100100 100        |        |            |
|    | -RG2 = | 1,10011011 100        |        |            |
|    | SM =   | <u>0,10110101 000</u> |        |            |
|    | SM =   | <u>0,01010000 100</u> | RG1← = | 0,00000001 |
| 2) | RG2→=  | 0,00110010 010        |        |            |
|    | -RG2 = | 1,11001101 110        |        |            |
|    | SM =   | <u>0,01010000 100</u> |        |            |
|    | SM =   | <u>0,00011110 010</u> | RG1← = | 0,00000011 |
| 3) | RG2→=  | 0,00011001 001        |        |            |
|    | -RG2 = | 1,11100110 111        |        |            |
|    | SM =   | <u>0,00011110 010</u> |        |            |
|    | SM =   | <u>0,00000101 001</u> | RG1← = | 0,00000111 |
| 4) | RG2→=  | 0,00001100 100        |        |            |
|    | -RG2 = | 1,11110011 100        |        |            |
|    | SM =   | <u>0,00000101 001</u> |        |            |
|    | SM =   | <u>1,11111000 101</u> | RG1← = | 0,00001110 |

$$\begin{array}{ll}
5) \quad \text{RG2} \rightarrow = \underline{0,00000110|010} & \text{RG1} \leftarrow = 0,00011100 \\
\quad \text{SM} = \underline{1,11111110|111} & \\
6) \quad \text{RG2} \rightarrow = \underline{0,00000011|001} & \text{RG1} \leftarrow = 0,00111001 \\
\quad \text{SM} = \underline{0,00000010|000} & \\
7) \quad \text{RG2} \rightarrow = \underline{0,00000001|100} & \\
\quad - \text{RG2} = \underline{1,11111110|100} & \\
\quad \text{SM} = \underline{0,00000010|000} & \text{RG1} \leftarrow = 0,01110011 \\
\quad \text{SM} = \underline{0,00000000|100} & \\
8) \quad \text{RG2} \rightarrow = \underline{0,00000000|110} & \\
\quad - \text{RG2} = \underline{1,11111110|100} & \\
\quad \text{SM} = \underline{0,00000000|100} & \text{RG1} \leftarrow = 0,11100111 \\
\quad \text{SM} = \underline{0,00000001|000} & \\
\\
\text{pA} = 00101 \quad (5) & \\
\text{-pB} = 00001 \quad (1) & \\
\text{pC} = 00110 \quad (6) &
\end{array}$$

Ответ: mC = 0,11100111    pC = 00110.

### 2.3. РАЗРАБОТКА ОБЩЕЙ СТРУКТУРЫ УСТРОЙСТВА, ВЫПОЛНЯЮЩЕГО ОПЕРАЦИЮ ДЕЛЕНИЯ

Полная структура устройства (рис. 7), выполняющего операцию деления по приведенному алгоритму, должна содержать часть, отвечающую за работу с мантиссами, и часть, отвечающую за работу с порядками, триггер для хранения знака результата.

Часть, отвечающая за работу с мантиссами, содержит сумматор мантисс SM, регистры мантисс RM1, RM2, триггер для хранения знака мантиссы Т и счетчик циклов СТ.

При реализации операции деления  $C = A/B$  в сумматор SM загружается мантисса делимого (mA), в регистр RM2 загружается мантисса делителя, регистр RM1 обнуляется, в нем будет формироваться мантисса частного (mC).

Сигналы с первых (знаковых) разрядов SM и RM2 подключаются к логическому элементу «сумма по модулю два», выход элемента подключен ко входу триггера. Данная схема позволяет вычислить знак результата на основе знаков операндов и сохранить его в триггере. После сохранения знака результата в триггере знаки операндов можно обнулить.

Сумматор SM и регистр RM2 имеют индикаторы нуля. При  $SM = 0$  формируется логическое условие  $x_1$ , при  $RM2 = 0$  формируется логическое условие  $x_2$ .

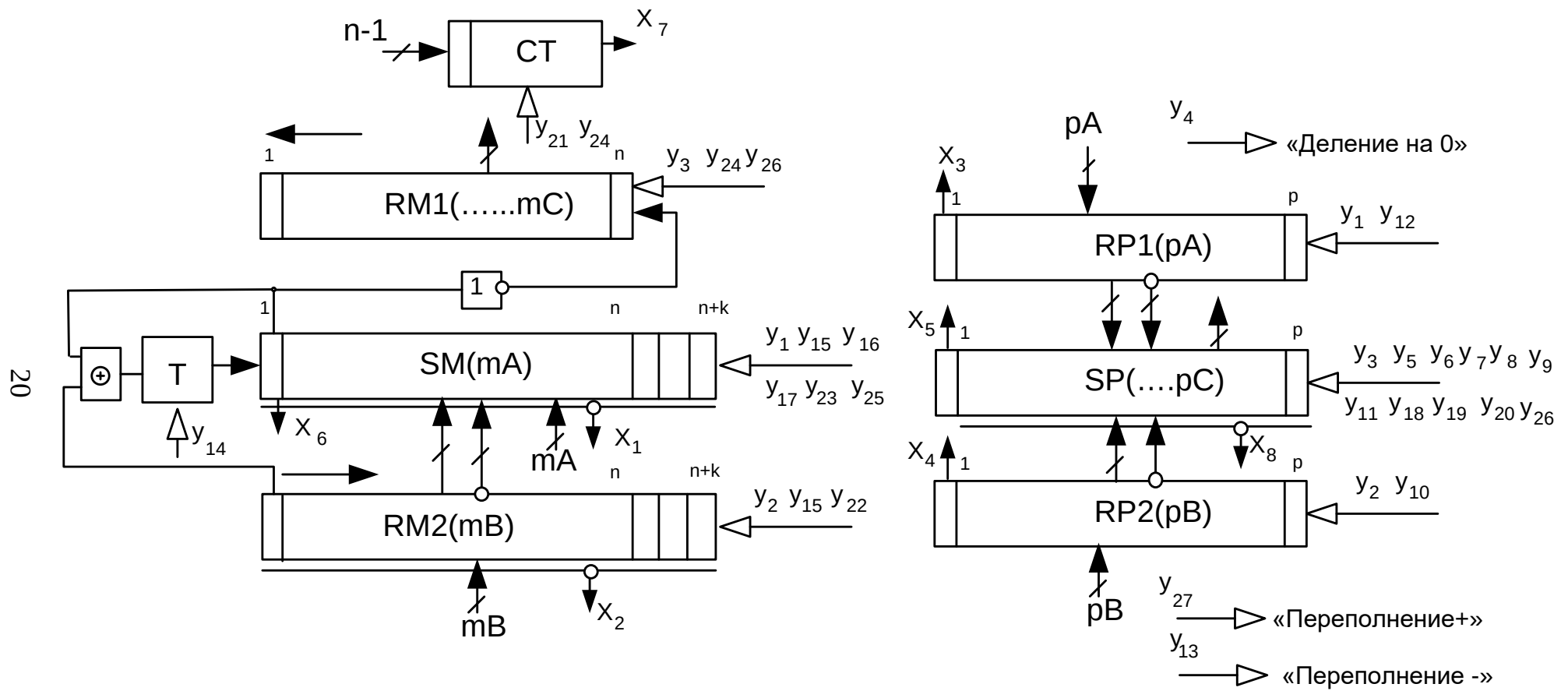


Рис. 7. Полная структурная схема устройства деления чисел с плавающей запятой

От регистра RM2 к сумматору SM идут две многоразрядные связи – прямая и инверсная (предполагается подключение через блок инверторов). Прямая связь обеспечивает операцию  $SM = SM + RM2$ , инверсная – операцию  $SM = SM - RM2$ , которая реализуется как

$$SM = SM + \overline{RM2} + 2^{-n}.$$

Счетчик СТ используется для подсчета необходимого числа циклов, на счетчик подаются два управляющих сигнала. Сигнал  $y_{21}$  загружает необходимое число циклов, соответствующее числу разрядов числа, за вычетом знакового (то есть в счетчик загружается  $n-1$ ). Сигнал  $y_{24}$  производит декремент счетчика (уменьшение на 1). Когда счетчик обнуляется, активизируется логическое условие  $x_7$ .

Часть, отвечающая за работу с порядками, содержит сумматор порядков SP, регистры порядков RP1 и RP2.

Сумматор порядков SP содержит индикатор нуля, настроенный на проверку всех разрядов, за исключением знакового (за исключением первого), когда все эти разряды равны 0 активизируется логическое условие  $x_8$ .

К сумматору SP от регистров RP1 и RP2 идут соответственно по две многоразрядные связи (прямая и инверсная), таким образом обеспечиваются реализация операций  $SP = SP + RP1$ ,  $SP = SP - RP1$ ,  $SP = SP + RP2$ ,  $SP = SP - RP2$ , а также операции загрузки  $SP = RP1$ ,  $SP = RP2$ .

Кроме того, устройство вырабатывает сигналы исключительных ситуаций: «деление на 0», «Переполнение +», «Переполнение –».

## **2.4. РАЗРАБОТКА АЛГОРИТМА ФУНКЦИОНИРОВАНИЯ УСТРОЙСТВА, ВЫПОЛНЯЮЩЕГО ОПЕРАЦИЮ ДЕЛЕНИЯ**

На рис. с 8 по 11 показана схема всего алгоритма выполнения операции деления с неподвижным сумматорам чисел с плавающей запятой в прямых кодах.

На рис. 8 приводится схема первого этапа работы алгоритма деления (ввод данных и начальная проверка).

Сначала производится загрузка данных, мантисса А загружается в сумматор мантисс SM, порядок в RP1. Мантисса В загружается в регистр мантисс RM2, порядок в RP2. Как правило, операнды загружаются с одной шины устройства, поэтому загрузку операндов А и В нужно разделить во

времени и управлять с помощью различных сигналов ( $y_1$  и  $y_2$ ). Порядок и мантиссу можно загружать в устройство одновременно, поскольку загрузка происходит в разные регистры по разным шинам. В соответствии со структурой устройства сигнал  $y_1$  нужно подавать к сумматору мантисс SM и к регистру порядка RP1, сигнал  $y_2$  – к регистрам RM2 и RP2.

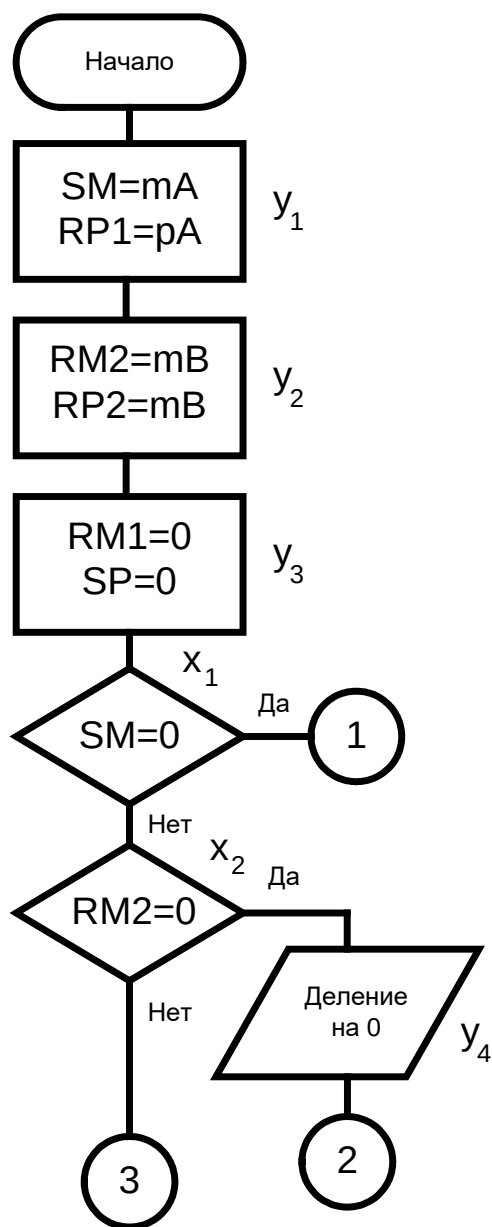


Рис. 8. Ввод данных и начальная проверка

Следующее действие – одновременно обнуляется регистр RM1, где будет формироваться мантисса результата, и сумматор порядков SP, где будет формироваться порядок результата, соответственно сигнал  $y_3$  подается на RM1 и SP.

При реализации длинных операций рекомендуется выполнять проверку операндов на ноль, во-первых, это позволяет в некоторых случаях сократить время выполнения операции, в некоторых случаях избежать зацикливания. Для реализации подобной проверки на ноль в структуре сумматора SM и регистра RM2 содержатся схемы-детекторы нуля, реализованные в простейшем случае на базе многовходового элемента ИЛИ.

Сначала выполняется проверка SM на ноль, и в случае, если проверка подтверждается, осуществляется переход на вывод результата (в регистрах RM1 и SP находятся нулевые значения, они и выведутся.) Далее проверяется

RM2 на ноль, если проверка подтверждается, выводится сообщение об ошибке «деление на ноль».

Далее на рис. 9 приводится схема алгоритма работы с порядками. При выполнении операции деления в формате с плавающей запятой порядок результата находится как  $pC = pA - pB$ , но поскольку порядки – целые

числа, представленные в прямом коде, это действие разворачивается в довольно большой алгоритм. В зависимости от знаков операндов (соответственно первые разряды регистров RP1 и RP2 – логические условия  $x_3$  и  $x_4$ ) с помощью оператора CASE рассматриваются все возможные варианты (их всего четыре).

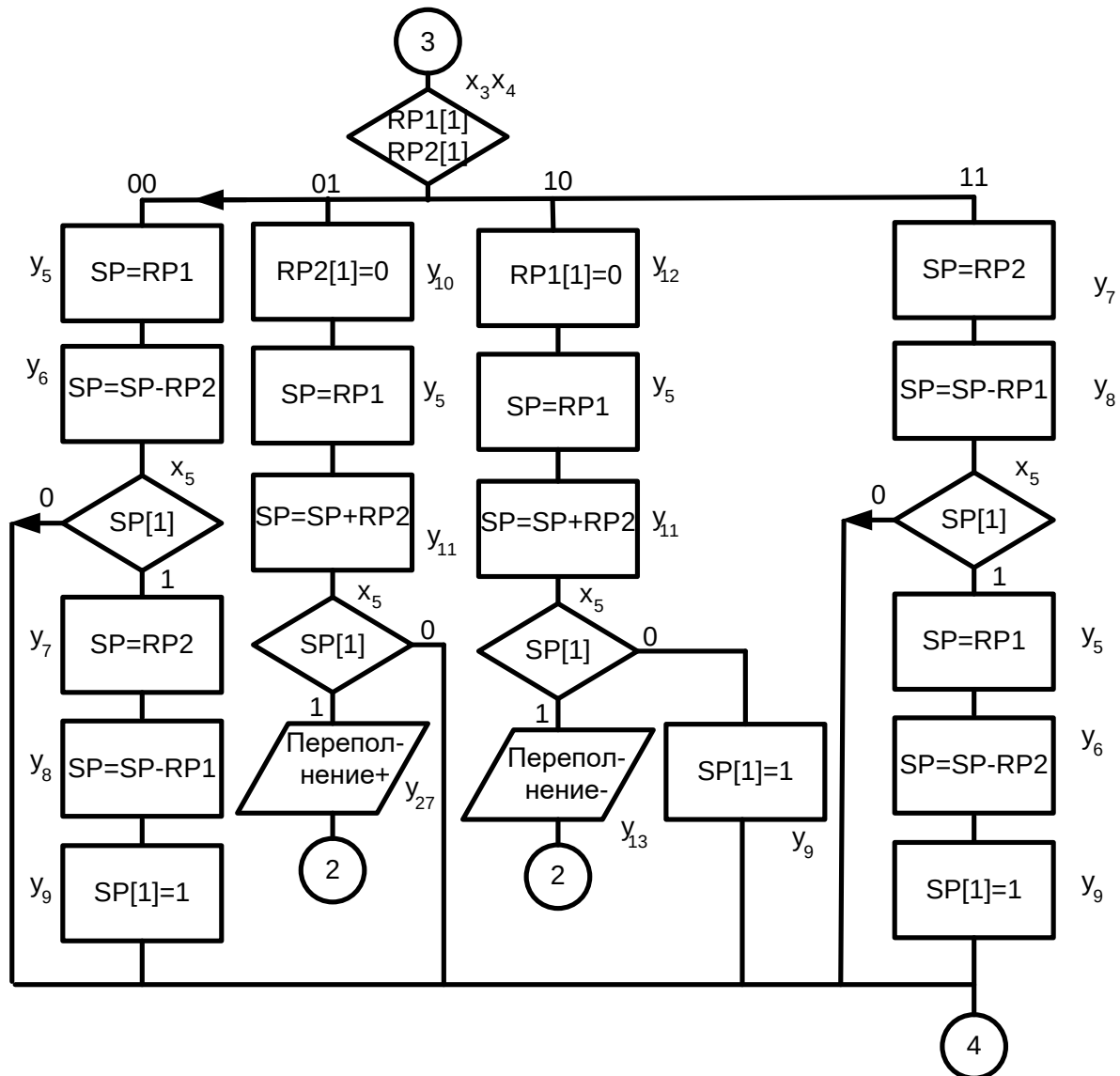


Рис. 9. Работа с порядками

Ветвь «00» – порядки положительны. Находим разность RP1 и RP2, причем в два этапа: на первом этапе содержимое RP1 загружается в сумматор порядков SP, на втором этапе производится вычитание SP и RP2 (то есть выполняется  $SP = SP + \overline{RP2} + 1$ ). Результат может получиться отрицательный, и если он отрицательный, то будет представлен в дополнительном коде, а поскольку в нашем случае используется прямой код, то легче заново пересчитать результат. Если знаковый разряд SP (условие  $x_5$ ) равен 1

(результат отрицательный), пересчитываем порядок: загружаем в SP уже RP2, находим разность с RP1 и принудительно присваиваем знаковый разряд «1», таким образом, получаем отрицательное число в прямом коде.

Ветвь «01» – порядок в RP1 положительный, в RP2 – отрицательный, то есть для того, чтобы найти разность RP1 и RP2, к RP1 прибавляется  $-RP2$ , которое в данном случае получается занулением знакового разряда RP2, то есть находим сумму двух положительных чисел. После сложения на сумматоре анализируется знак результата на предмет переполнения, если знак окажется «1», выдается сообщение о переполнении («переполнение +»), и работа алгоритма завершается, если переполнения нет, работа алгоритма продолжается.

Ветвь «10» – порядок в RP2 положительный, в RP1 – отрицательный. Здесь аналогично, чтобы найти разность RP2 и RP1, к RP2 прибавляется  $-RP1$ , которое получается занулением знакового разряда RP1, находим сумму двух положительных чисел, при этом зная, что они отрицательные. После сложения на сумматоре анализируется знак результата на предмет переполнения, если знак окажется «1», выдается сообщение о переполнении («переполнение –»), и работа алгоритма завершается. Рекомендуется различать положительное переполнение порядков (машинная бесконечность) и отрицательное переполнение порядков (машинный ноль), во втором случае результат операции можно просто считать нулевым (на самом деле он просто настолько мал, что не фиксируется заданной разрядной сеткой). В случае отсутствия переполнения принудительно присваивается отрицательный знак результату порядков.

Ветвь «11» – порядки отрицательны. Находим разность RP2 и RP1, причем в два этапа: на первом этапе содержимое RP2 загружается в сумматор порядков SP, на втором этапе производится вычитание SP и RP1 (то есть выполняется  $SP = SP + \overline{RP1} + 1$ ). Результат может получиться отрицательный, и если он отрицательный, то будет представлен в дополнительном коде, а поскольку в нашем случае, используется прямой код, то легче заново пересчитать результат. Если знаковый разряд SP (условие  $x_5$ ) равен 1 (результат отрицательный, знаковые разряды компенсируют друг друга), пересчитываем порядок: загружаем в SP уже RP1, находим разность с RP2 и принудительно присваиваем знаковый разряд «1», таким образом, получаем отрицательное число в прямом коде.

Далее (рис. 10) в триггер загружается знак мантиссы результата, вычисляемой на основе знаков операндов с помощью элемента, реализующего функцию «сумма по модулю два», после чего знаковые разряды операндов в SM и RM2 обнуляются.

На следующем этапе при необходимости производится денормализация делимого.

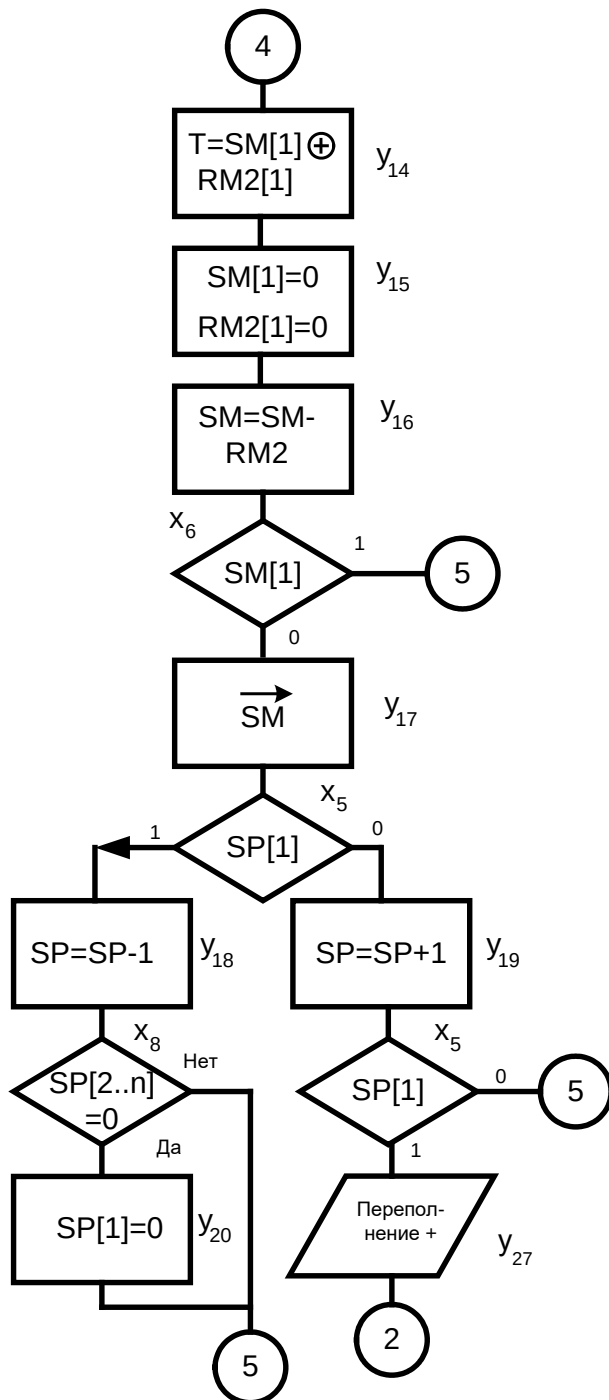


Рис. 10. Денормализация делимого

при отрицательном в положительный, при таком переходе в прямых кодах необходимо принудительно установить знак порядка в 0.

С целью обеспечения в результате нормализованного частного необходимо, чтобы соблюдалось условие  $|mA| < |mB|$ . Для проверки этого из сумматора мантисс вычитается RM2 и анализируется знак результата, если результат отрицательный (логическое условие  $x_6$ ), условие  $|mA| < |mB|$  выполняется и осуществляется переход непосредственно к выполнению алгоритма деления над мантиссами, при этом восстанавливать сумматор нет необходимости, поскольку далее используется алгоритм деления без восстановления остатка. Если условие  $|mA| < |mB|$  не выполняется ( $x_6 = 0$ ), производится денормализация делимого на сумматоре SM, сумматор SM сдвигается в сторону младших разрядов, а порядок на SP увеличивается на единицу. При этом, если порядок положительный, он увеличивается на единицу и отслеживается возможное переполнение порядков. Если порядок отрицательный, он уменьшается на единицу, отслеживается возможный случай перехода порядка из отрицательного в положительный, при таком переходе в прямых кодах необходимо принудительно установить знак порядка в 0.

Собственно алгоритм деления мантисс показан на рис. 11.

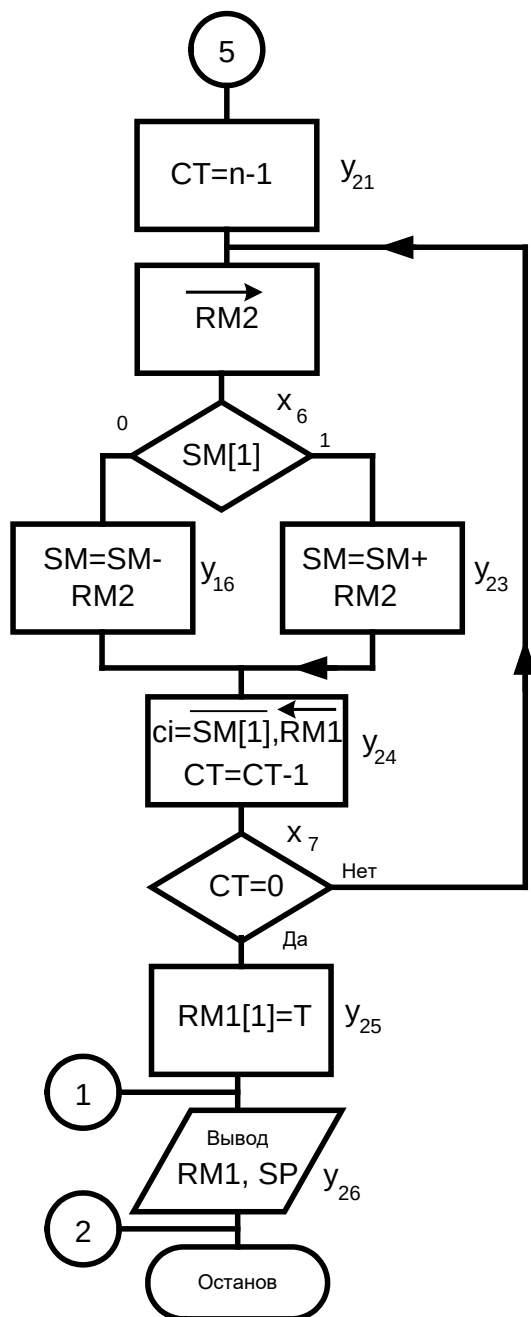


Рис. 11. Деление мантисс и вывод

Сначала в счетчик СТ заносится необходимое число циклов, соответствующее количеству цифр частного, поскольку один разряд отводится под знак, получается  $n-1$  цикл.

Далее выполняется непосредственно алгоритм деления мантисс.

Поскольку сумматор мантисс остается неподвижным, сдвигается регистр RM2, содержащий делимое.

При выполнении алгоритма без восстановления остатка частичный остаток на сумматоре может быть как положительный, так и отрицательный, поэтому сначала определяется знак сумматора SM. Если он положительный ( $x_6 = 0$ ), производится вычитание сдвинутого делителя на RM2 из сумматора. Если знак сумматора отрицательный ( $x_6 = 1$ ), производится сложение сдвинутого делителя на RM2 и сумматора. В любом случае инверсия знака на сумматоре (это и есть очередной разряд частного) задвигается в регистр RM1 со стороны младших разрядов.

Далее содержимое счетчика уменьшается на 1, и проверяется на ноль содержимое счетчика. Если, счетчик не равен нулю ( $x_7 = 0$ ), начинается новый цикл. Если счетчик равен нулю ( $x_7 = 1$ ), то есть все разряды частного получены, присваивается знак результата, сохраненный в триггере, и выводится результат. Мантисса результата находится в регистре RM1, порядок на сумматоре порядков SP.

Алгоритм заканчивается.

Сведем в одну таблицу все логические условия, формируемые устройством (табл. 1).

Таблица 1

**Логические условия**

| Обозначение | Сокращенная запись | Комментарии                                                       |
|-------------|--------------------|-------------------------------------------------------------------|
| $x_1$       | SM=0               | Проверка сумматора мантисс (делимого) на ноль                     |
| $x_2$       | RM2=0              | Проверка регистра делителя на ноль                                |
| $x_3$       | RP1[1]             | Знак порядка делимого                                             |
| $x_4$       | RP2[1]             | Знак порядка делителя                                             |
| $x_5$       | SP[1]              | Знак сумматора порядка                                            |
| $x_6$       | SM[1]              | Знак сумматора мантисс                                            |
| $x_7$       | CT=0               | Проверка счетчика на ноль                                         |
| $x_8$       | SP[2...p]          | Проверка сумматора порядков на ноль (без учета знакового разряда) |

Сведем в одну таблицу все управляющие сигналы, формируемые устройством (табл. 2).

Таблица 2

**Управляющие сигналы**

| Обозначение | Сокращенная запись        | Комментарии                                                     |
|-------------|---------------------------|-----------------------------------------------------------------|
| $y_1$       | SM = mA, RP1 = pA         | Загрузка мантиссы и порядка А                                   |
| $y_2$       | RM2 = mB, RP2 = pB        | Загрузка мантиссы и порядка В                                   |
| $y_3$       | RM1 = 0, SP = 0           | Сброс RM1 и SP                                                  |
| $y_4$       | Деление на 0              | Вывод сигнала «Деление на 0»                                    |
| $y_5$       | SP = RP1                  | Загрузка сумматора SP из RP1                                    |
| $y_6$       | SP = SP – RP2             | Вычитание SP и RP2, результат в SP (содержимое RP2 не меняется) |
| $y_7$       | SP = RP2                  | Загрузка сумматора SP из RP2                                    |
| $y_8$       | SP = SP – RP1             | Вычитание SP и RP1, результат в SP (содержимое RP1 не меняется) |
| $y_9$       | SP[1] = 1                 | Установка знакового разряда SP                                  |
| $y_{10}$    | RP2[1] = 0                | Сброс знакового разряда RP2                                     |
| $y_{11}$    | SP = SP + RP2             | Сумма SP и RP2, результат в SP (содержимое RP2 не меняется)     |
| $y_{12}$    | RP1[1] = 0                | Сброс знакового разряда RP1                                     |
| $y_{13}$    | Переполнение –            | Вывод сигнала «Переполнение –»                                  |
| $y_{14}$    | T = SM[1] $\oplus$ RM2[1] | Получение знака результата и сохранение его в триггере          |

| Обозначение | Сокращенная запись                       | Комментарии                                       |
|-------------|------------------------------------------|---------------------------------------------------|
| $y_{15}$    | $SM[1] = 0, RM2[1] = 0$                  | Сброс знаковых разрядов SM и RM2                  |
| $y_{16}$    | $SM = SM - RM2$                          | Вычитание SM и RM2, результат в SM                |
| $y_{17}$    | $SM \rightarrow$                         | Сдвиг SM вправо                                   |
| $y_{18}$    | $SP = SP + 1$                            | Увеличение SP на 1                                |
| $y_{19}$    | $SP = SP - 1$                            | Уменьшение SP на 1                                |
| $y_{20}$    | $SP[1] = 0$                              | Сброс знакового разряда SP                        |
| $y_{21}$    | $CT = n - 1$                             | Загрузка счетчика (n-1 циклов)                    |
| $y_{22}$    | $RM2 \rightarrow$                        | Сдвиг RM2 вправо                                  |
| $y_{23}$    | $SM = SM + RM1$                          | Сложение SM и RM2, результат в SM                 |
| $y_{24}$    | $c_i = \overline{SM[1]}, RM1 \leftarrow$ | В RM1 задвигается очередной разряд частного $c_i$ |
| $y_{25}$    | $RM1[1] = T$                             | Присвоение знака результата                       |
| $y_{26}$    | Вывод RM1, SP                            | Вывод мантиссы и порядка результата               |
| $y_{27}$    | Переполнение +                           | Вывод сигнала «Переполнение +»                    |

### 3. ОБЩИЕ РЕКОМЕНДАЦИИ К ВЫПОЛНЕНИЮ РАСЧЕТНО-ГРАФИЧЕСКОЙ РАБОТЫ

Расчетно-графическая работа является заключительным этапом в изучении дисциплины «Арифметические и логические основы вычислительных систем». Цель выполнения работы – закрепить и расширить теоретические знания студента, выработать у него навыки применения программных средств для решения задач по проектированию арифметических устройств.

Основной задачей расчетно-графической работы является разработка операционного устройства на структурном уровне, выполняющего заданную арифметическую операцию. При этом предполагается подробная проработка алгоритма работы устройства и, факультативно, моделирование работы устройства посредством языка высокого уровня.

Расчетно-пояснительная записка должна включать следующие разделы:

1. Общее описание алгоритма выполнения заданной арифметической операции, упрощенный пример выполнения арифметической операции (делается вручную).

2. Разработка структуры операционного устройства, детальная разработка алгоритма работы выполнения операции.

3. Моделирование работы операционного устройства средствами языка высокого уровня, разработка соответствующей программы (желательно).

4. Тестовые примеры.

Графическая часть:

- структурная схема разрабатываемого устройства;
- схема алгоритма работы устройства.

Желательно разработанную структуру устройства и алгоритм снабдить подробными пояснениями.

В табл. 3 приводятся варианты расчетно-графических работ. В графе «код» указывается прямой, обратный или дополнительный код для двоичных чисел, для двоично-десятичных указывается D-код: D8421 или D8421+3. Для двоичных чисел предполагается формат представления чисел с плавающей запятой, для двоично-десятичных – формат с фиксированной запятой.

Ускоренные алгоритмы выполнения арифметических операций предполагают анализ двух разрядов одновременно (2p) или трех разрядов одновременно (3p).

## Варианты расчетно-графических работ

| Вариант | Операция  | Код            | Модифицированный | Алгоритм                                         | Ускорение |
|---------|-----------|----------------|------------------|--------------------------------------------------|-----------|
| 1       | умножение | прямой         | нет              | Младшими разрядами вперед                        | нет       |
| 2       | умножение | обратный       | нет              | Младшими разрядами вперед                        | нет       |
| 3       | умножение | дополнительный | нет              | Младшими разрядами вперед                        | нет       |
| 4       | умножение | прямой         | нет              | Старшими разрядами вперед                        | нет       |
| 5       | умножение | обратный       | нет              | Старшими разрядами вперед                        | нет       |
| 6       | умножение | дополнительный | нет              | Старшими разрядами вперед                        | нет       |
| 7       | умножение | прямой         | да               | Младшими разрядами вперед                        | нет       |
| 8       | умножение | обратный       | да               | Младшими разрядами вперед                        | нет       |
| 9       | умножение | дополнительный | да               | Младшими разрядами вперед                        | нет       |
| 10      | умножение | прямой         | да               | Младшими разрядами вперед                        | 2р        |
| 11      | умножение | дополнительный | да               | Младшими разрядами вперед                        | 2р        |
| 12      | умножение | прямой         | да               | Младшими разрядами вперед                        | 3р        |
| 13      | умножение | дополнительный | да               | Младшими разрядами вперед                        | 3р        |
| 14      | деление   | прямой         | нет              | Со сдвигом сумматора, с восстановлением остатка  | нет       |
| 15      | деление   | прямой         | нет              | Со сдвигом сумматора, без восстановления остатка | нет       |
| 16      | деление   | дополнительный | нет              | Со сдвигом сумматора, без восстановления остатка | нет       |
| 17      | деление   | прямой         | да               | Со сдвигом сумматора, без восстановления остатка | нет       |
| 18      | деление   | дополнительный | да               | Со сдвигом сумматора, без восстановления остатка | нет       |

| Вариант | Операция                     | Код            | Модифицированный | Алгоритм                                             | Ускорение |
|---------|------------------------------|----------------|------------------|------------------------------------------------------|-----------|
| 19      | деление                      | прямой         | нет              | С неподвижным сумматором, с восстановлением остатка  | нет       |
| 20      | деление                      | прямой         | нет              | С неподвижным сумматором, без восстановления остатка | нет       |
| 21      | деление                      | прямой         | да               | Со сдвигом сумматора, без восстановления остатка     | 2р        |
| 22      | деление                      | дополнительный | да               | Со сдвигом сумматора, без восстановления остатка     | 2р        |
| 23      | деление                      | прямой         | да               | Со сдвигом сумматора, без восстановления остатка     | 3р        |
| 24      | деление                      | дополнительный | да               | Со сдвигом сумматора, без восстановления остатка     | 3р        |
| 25      | извлечение квадратного корня | прямой         | нет              |                                                      | нет       |
| 26      | извлечение квадратного корня | прямой         | нет              |                                                      | 2р        |
| 27      | сложение                     | прямой         | нет              |                                                      | нет       |
| 28      | сложение                     | прямой         | да               |                                                      | нет       |
| 29      | извлечение квадратного корня | дополнительный | нет              |                                                      | нет       |
| 30      | извлечение квадратного корня | дополнительный | нет              |                                                      | 2р        |
| 31      | сложение                     | дополнительный | да               |                                                      | нет       |
| 32      | сложение                     | дополнительный | да               |                                                      | нет       |
| 33      | вычитание                    | прямой         | нет              |                                                      | нет       |
| 34      | вычитание                    | прямой         | да               |                                                      | нет       |
| 35      | вычитание                    | обратный       | нет              |                                                      | нет       |
| 36      | вычитание                    | дополнительный | нет              |                                                      | нет       |

| Вариант | Операция  | Код            | Модифицированный | Алгоритм                                             | Ускорение |
|---------|-----------|----------------|------------------|------------------------------------------------------|-----------|
| 37      | вычитание | дополнительный | да               |                                                      | нет       |
| 38      | умножение | D8421          | нет              | Младшими разрядами вперед                            | нет       |
| 39      | умножение | D8421          | нет              | Старшими разрядами вперед                            | нет       |
| 40      | умножение | D8421+3        | нет              | Младшими разрядами вперед                            | нет       |
| 41      | умножение | D8421+3        | нет              | Старшими разрядами вперед                            | нет       |
| 42      | деление   | D8421          | нет              | Со сдвигом сумматора, с восстановлением остатка      | нет       |
| 43      | деление   | D8421          | нет              | Со сдвигом сумматора, без восстановления остатка     | нет       |
| 44      | деление   | D8421          | нет              | С неподвижным сумматором, с восстановлением остатка  | нет       |
| 45      | деление   | D8421          | нет              | С неподвижным сумматором, без восстановления остатка | нет       |
| 46      | деление   | D8421+3        | нет              | Со сдвигом сумматора, с восстановлением остатка      | нет       |
| 47      | деление   | D8421+3        | нет              | Со сдвигом сумматора, без восстановления остатка     | нет       |
| 48      | деление   | D8421+3        | нет              | С неподвижным сумматором, с восстановлением остатка  | нет       |
| 49      | деление   | D8421+3        | нет              | С неподвижным сумматором, без восстановления остатка | нет       |

## БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Потапов, В. И. Компьютерная арифметика и алгоритмическое моделирование арифметических операций [Электронный ресурс] : учеб. пособие для студентов по направлению 230100 "Информатика и вычислительная техника" / В. И. Потапов, О. П. Шафеева ; ОмГТУ. - Электрон. текстовые дан. (936 Кб). – Омск : Изд-во ОмГТУ, 2014. – 1 эл. опт. диск (CD-ROM).

2. Потапов, В. И. Компьютерная арифметика и алгоритмическое моделирование арифметических операций : учеб. пособие для вузов по направлению 230100 «Информатика и вычислительная техника» / В. И. Потапов, О. П. Шафеева. – Омск : Изд-во ОмГТУ, 2005. – 95 с.

3. Потапов, В. И. Основы компьютерной арифметики и логики : учеб. пособие для вузов по направлению 230100 «Информатика и вычислительная техника» / В. И. Потапов, О. П. Шафеева, И. В. Червенчук. – Омск : Изд-во ОмГТУ, 2004. – 172 с.